

Specifikacija Projekta

1. Tim

Luka Marić RA154/2022

2. Zadatak

Za ciljnu ocenu 10 projekat obuhvata:

- Implementaciju generičkog aktorskog radnog okvira u programskom jeziku Go, sa osnovnim i dodatnim funkcionalnostima (supervizija aktora, middleware)
- Implementaciju sistema za federativno učenje nad aktorskim okvirom, primenjenog na problem detekcije mrežnih napada (intrusion detection), čime se demonstrira očuvanje privatnosti mrežnog saobraćaja između organizacija
- Podršku za dva režima rada: **provider klaster** (centralizovani koordinator) i **peer-to-peer** (decentralizovana agregacija), sa izborom prilikom pokretanja
- Upotrebu **CRDT** struktura za konzistentno praćenje stanja sistema u distribuiranom okruženju

3. Federativno učenje

3.1 Problem koji se rešava

Više organizacija (npr. kompanija, ISP-ova, data centara) želi da trenira zajednički model za klasifikaciju mrežnog saobraćaja na normalan i različite tipove napada (DoS, Probe, R2L, U2R). Nijedna organizacija ne želi da deli svoj sirovi mrežni saobraćaj sa ostalima jer on otkriva internu topologiju mreže, ranjivosti, i poverljive informacije o korisnicima. Federativno učenje omogućava treniranje zajedničkog modela bez centralizacije podataka. Svaka organizacija trenira lokalno i deli samo ažuriranja modela (gradijente/težine).

3.2 Algoritam

Federated Averaging (FedAvg) - McMahan et al., 2017.

Postupak po rundi:

1. Koordinator (ili peer inicijator) šalje trenutni globalni model svim učesnicima
2. Svaki učesnik trenira model lokalno na svojim podacima kroz E lokalnih epoha
3. Učesnik šalje ažurirane težine modela Agregatoru (provider režim) ili svim susedima (P2P režim)
4. Agregator primenjuje ponderisani prosek (provider režim), odnosno svaki peer lokalno agregira primljene težine i konvergira ka globalnom modelu (P2P režim)
5. Ponavljanje za R rundi

Model: Višeslojna potpuno povezana neuronska mreža (MLP):

- Ulazni sloj: ~122 dimenzije (41 NSL-KDD feature; 3 kategorijska — `protocol_type`, `service`, `flag` — se one-hot enkodiraju, pa je dimenzija veća od 41)
- Skriveni sloj 1: 64 neurona, ReLU aktivacija
- Skriveni sloj 2: 32 neurona, ReLU aktivacija
- Izlazni sloj: 5 klasa (Normal, DoS, Probe, R2L, U2R), softmax

3.3 Skup podataka

NSL-KDD

- Trening set: ~125.000 zapisa
- Test set: ~22.500 zapisa
- 41 numerički i kategorički feature
- 5 klasa: Normal, DoS, Probe, R2L, U2R

3.4 Distribucija podataka

- **IID**: nasumična jednolika raspodela zapisa među učesnicima
- **Non-IID**: svaki učesnik ima dominantnu klasu saobraćaja (simulira specijalizaciju organizacija)

3.5 Evaluacija rezultata

- **Metrike**: Accuracy, Precision/Recall/F1 po klasi, Confusion matrix
- **Konvergencija**: tačnost i macro-F1 po rundi (strukturirani log) za oba režima

4. Aktorski sistem

4.1 Vrste aktora i uloge

Aktor	Uloga	Režim
Coordinator	Upravlja rundama, distribuira globalni model, prima rezultate agregacije, pokreće evaluaciju nakon svake runde	Provider
Aggregator	Prima težine od Trainera, prati ko je poslao update za tekuću rundu, računa ponderisani prosek, vraća novi globalni model Coordinatoru	Provider
Trainer	Učitava lokalne podatke, trenira model lokalno, šalje ažurirane težine Aggregatoru; čuva poslednju završenu rundu i težine radi idempotentnosti	Oba
Evaluator	Prima globalni model i test set, računa metrike, prijavljuje rezultate	Oba
Logger	Prima log poruke od svih aktora, formatira i zapisuje u strukturirani log	Oba
PeerCoordinator	Pokreće runde, prikuplja težine od suseda, lokalno agregira, propagira rezultat; sinhronizuje CRDT stanje sa ostalim peer-ovima	P2P

U provider režimu aktori su: Coordinator, Aggregator, Trainer, Evaluator, Logger.

U P2P režimu: svaki čvor ima PeerCoordinator + Trainer + Evaluator + Logger (nema centralnog Aggregatora).

Hijerarhija i pokretanje:

- Coordinator pri pokretanju kreira Agregator i Evaluator kao svoju decu (i postaje njihov supervisor).
- Traineri se pokreću zasebno (različiti procesi/mašine); adresa Coordinadora se prosleđuje kao konfiguracija (env `COORDINATOR_ADDR`, oblik `actor://host:port/coordinator`).
- Logger se kreira pri pokretanju i supervizuje se od strane Coordinadora.

4.2 Poruke

Poruka	Pošiljalac → Primalac	Sadržaj	Namena
RegisterTrainer	Trainer → Coordinator	{trainer_id, dataset_size, address}	Registracija učesnika na početku; Coordinator pokreće prvu rundu kada se registruje očekivani broj Trainera
StartRound	Coordinator → Trainer	{round_number, global_weights, aggregator_address}	Početak nove runde, distribucija modela i adrese Aggregatora
LocalUpdate	Trainer → Aggregator	{trainer_id, round_number, updated_weights, dataset_size, local_loss}	Slanje lokalnih ažuriranja nakon treninga
AggregationComplete	Aggregator → Coordinator	{round_number, new_global_weights}	Rezultat agregacije; Coordinator čuva novi globalni model u svom stanju
EvaluateModel	Coordinator → Evaluator	{round_number, weights, test_data_path}	Zahtev za evaluaciju modela koji se šalje nakon svake runde
EvaluationResult	Evaluator → Coordinator	{round_number, accuracy, class_metrics map[string]F1Score, confusion_matrix}	Rezultati evaluacije
TrainingComplete	Coordinator → Logger	{total_rounds, final_accuracy, duration}	Obaveštenje o završetku celokupnog treninga
LogEntry	* → Logger	{timestamp, source_actor, level, message}	Strukturirano logovanje događaja (fire-and-forget)
SupervisionAlert	Framework → roditelj (Coordinator)	{child, reason}	Obaveštenje o padu i restartu child aktera; roditelj može reagovati (npr. ponovo poslati StartRound)
PeerSync	PeerCoordinator → PeerCoordinator	{peer_id, round_number, local_weights, dataset_size, crdt_state}	Razmena lokalnih težina i sinhronizacija CRDT stanja između peer-ova

4.3 Napomene o pouzdanosti

Timeout mehanizam: Coordinator pokreće tajmer pri slanju `StartRound` . Ako do isteka roka ne stignu ažuriranja od svih Trainera, Coordinator nalaže Aggregatoru da rundu završi sa primljenim ažuriranjima (spori/pali Traineri se otpisuju za tu rundu). Ovo obezbeđuje da spori ili pali Traineri ne blokiraju sistem.

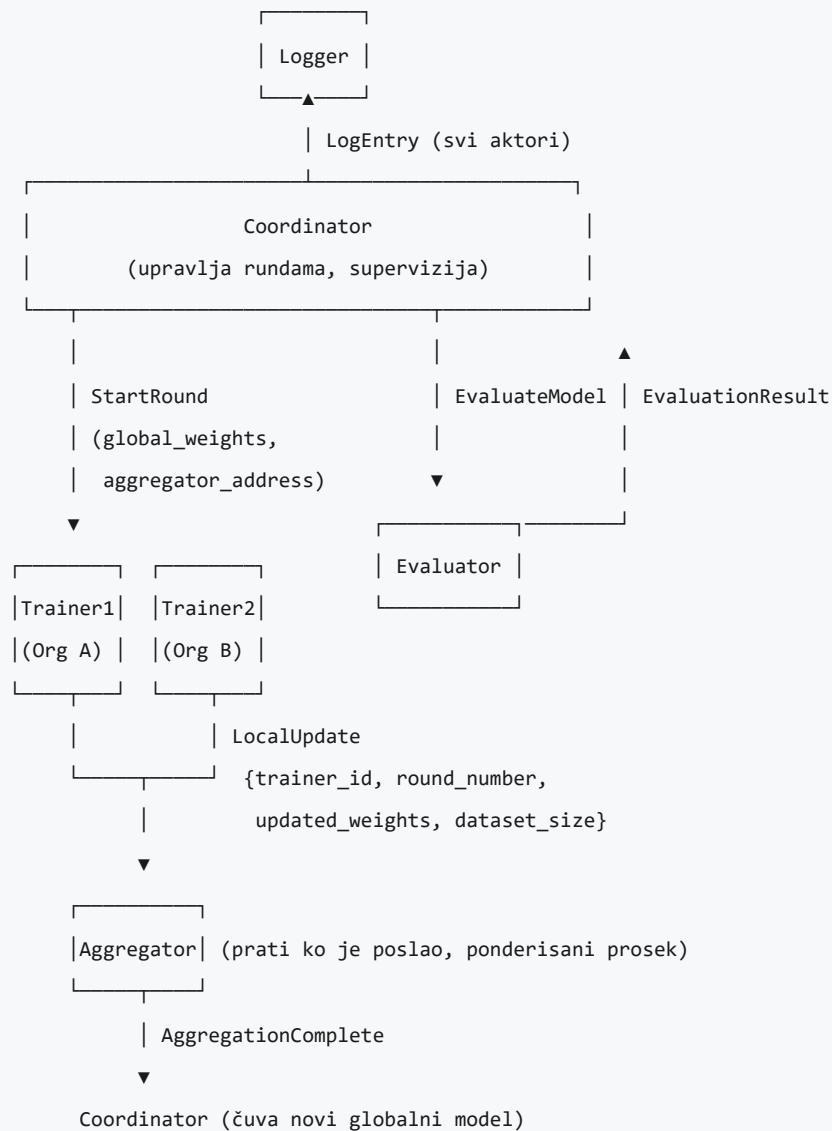
Idempotentnost Trainera: Trainer čuva poslednju završenu rundu i poslednje poslate težine. Ako primi `StartRound` sa istim `round_number` koji je već obradio (npr. zbog restarta Agregatora), ponovo šalje iste težine bez ponovnog treniranja.

Pad Agregatora usred runde: framework (supervizija roditelja) restartuje Agregator, a Coordinator dobija `SupervisionAlert` i ponovo šalje `StartRound` sa istim `round_number` svim Trainerima. Zahvaljujući idempotentnosti, Traineri samo prosleđuju već izračunate težine, pa sveže pokrenuti Agregator ponovo prikupi ažuriranja.

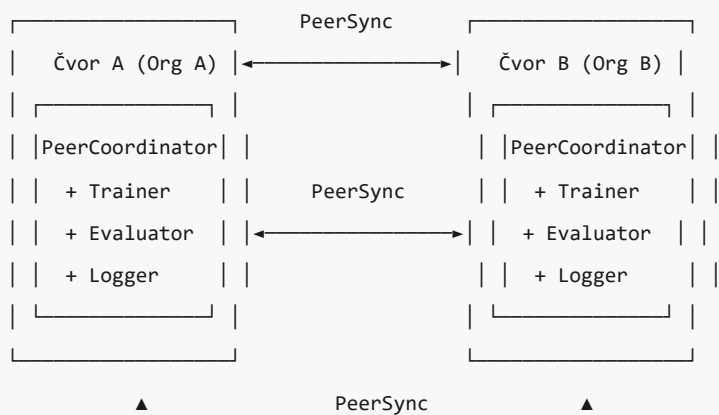
Pad Evaluatora između rundi: framework restartuje Evaluator; Coordinator dobija `SupervisionAlert` i samo to zabeleži (bez dodatnih akcija).

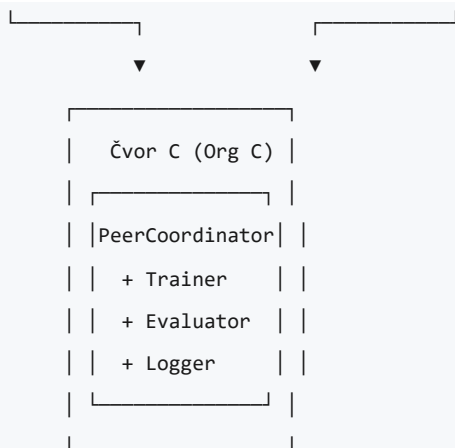
4.4 Skica komunikacije

=== PROVIDER REŽIM (zvezdasta topologija) ===



=== PEER-TO-PEER REŽIM (mesh topologija) ===





CRDT stanje (G-Counter, OR-Set) se sinhronizuje kroz PeerSync poruke pri svakoj komunikaciji.

4.5 CRDT upotreba

Za konzistentno praćenje distribuiranog stanja bez centralnog autoriteta koriste se sledeće CRDT strukture:

- **G-Counter** (Grow-only Counter): monotoni, konvergentni brojač ukupno odrađenih rundi u klasteru. Merge operacija uzima max po čvoru, a vrednost je zbir; nakon sinhronizacije svi peer-ovi imaju identičnu vrednost, bez centralnog autoriteta.
- **OR-Set** (Observed-Remove Set): praćenje skupa aktivnih učesnika. Podržava konkurentne join/leave (add/remove) operacije uz pravilo „add pobeđuje“, pa daje konzistentnu sliku učesnika u P2P režimu.

U provider režimu OR-Set se koristi za praćenje skupa registrovanih Trainera.

5. Aktorski radni okvir

- **Obavezni elementi:** Aktori, asinhronne poruke, Mailbox, Become, Lifecycle (PreStart/PostStop), Remote (gRPC)
- **Dodatni elementi:** Supervizija (restart strategija za pale aktore, notifikacija roditelju), Middleware (logovanje poruka, merenje latencije)

Objedinjeni ulaz `cmd/node` bira režim pri pokretanju: `--mode=provider --role=coordinator|trainer` ili `--mode=p2p` (uloga je uvek peer). Alternativno postoje i zasebni binari `cmd/coordinator`, `cmd/trainer`, `cmd/peer`. Demonstracija na više mašina putem gRPC remote aktora.

6. Tehnologije

Go 1.25+, gRPC + Protocol Buffers, Docker, `log/slog`. Neuronska mreža (MLP + backpropagation) i parsiranje NSL-KDD dataseta implementirani su ručno, bez eksternih ML ili CSV biblioteka.